PRESENTER TALK TRACK · 60 MINUTES

Lakebase 101

Suncorp AI CoE · CDP · Infosec — narration, timing, live-demo run-sheet, and Q&A prep for the 31-slide deck.

How to use this

Each entry below maps to one slide: **SAY** is the spoken narration (paraphrase, don't read verbatim), **CUE** is the click/transition or demo action. Suggested speaker in the top-right of each block — adjust freely. Total budget is 60 minutes including a 15-minute live demo; keep Q&A flowing throughout rather than saving it all for the end.

Timing plan

SECTION	SLIDES	LEAD	TIME
Open & agenda	1–2	Host	**2 min**
01 · Context	3–4	AI CoE (Kranthi)	**4 min**
02 · Lakebase 101	5–9	AI CoE	**8 min**
03 · Reference architecture — data flow	10–12	AI CoE + CDP	**8 min**
04 · Security & connectivity	13–18	Infosec + AI CoE	**12 min**
05 · Lakeflow cost controls	19–23	CDP	**5 min**
06 · Live demo (opens with branching)	24–28	SA / AI CoE	**15 min**
Wrap-up & next steps	29–30	Host	**4 min**
Q&A buffer	31	All	**2 min**
Total			**60 min**

Slide-by-slide narration

1 Lakebase 101 — title

Host · 0:30

SAY: Welcome. Over the next hour we'll cover what Lakebase is, where it fits in your claims architecture, how the identity and security model works, how you control cost, and then a live demo of a Databricks agent and a UiPath agent sharing memory. We built the agenda around the three lenses in the room — AI CoE, CDP, and Infosec — so each of you gets the part you care about.

CUE: Advance once introductions are done.

2 What we will cover

Host · 1:30

SAY: Six parts. Context and the problem; Lakebase fundamentals; the data-flow architecture; security and connectivity — that's the Infosec-heavy part; cost controls for the CDP team; and the live demo. Please ask questions as we go — this is a working session, not a webinar.

CUE: Note the ~15-min demo and that the deck is a shareable reference afterward.

3 Section 01 — Context

AI CoE · 0:20

SAY: Let's start with why we're spending an hour on an operational database.

CUE: Quick section divider — keep moving.

4 Why Lakebase, why now

AI CoE · 3:30

SAY: Agentic workflows behave differently from analytics. An agent reads and writes small pieces of state constantly, and it needs answers in milliseconds — that's an OLTP pattern, not a warehouse scan. In claims specifically, a UiPath bot doing document review and a Databricks triage agent need to act on the *same* claim, from the same state. Historically that means standing up a separate Postgres, outside your governance. Lakebase changes that: Postgres speed, but inside Unity Catalog. The goal today is to get all three teams aligned on one pattern we can take toward production.

CUE: Invite one sentence from Infosec on why "outside governance" has been the blocker.

5 Section 02 — Lakebase 101

AI CoE · 0:20

SAY: So what actually is Lakebase.

6 What is Lakebase

AI CoE · 4:00

SAY: Lakebase is fully-managed PostgreSQL — real Postgres, open wire protocol — built into Databricks. It's serverless and autoscaling from half a compute unit up to 32, and it scales to zero when idle, so you're not paying for a database that's sitting quiet overnight. It supports git-style branching, so you can spin up an instant copy for dev or test. And because it's part of the platform, it's integrated with Unity Catalog. Net: millisecond reads and writes for apps, agents, and serving — with governance attached.

CUE: If asked "is this just RDS?" — no: it's serverless, scales to zero, branches, and is UC-governed. Park deeper cost talk for §05.

7 The autoscaling model & where it fits

AI CoE · 3:30

SAY: The resource model is simple: a Project contains Branches, and each Branch has Endpoints — read-write and, if you want, read-only. It scales to zero and you pay for use. The most important idea on this slide is on the right: Lakebase is an **acceleration and serving layer, not your system of record**. Delta on Unity Catalog stays the source of truth. Lakebase is the low-latency mirror agents and apps talk to. Keep that boundary and the rest of the architecture falls into place.

CUE: This "not the SoR" line is the thesis — say it slowly. Transition: "so how does data get there?"

8 How autoscaling works

AI CoE · 2:30

SAY: A little more on how the autoscaling actually behaves, because it underpins both the performance and the cost story. Compute is measured in Compute Units, two gigabytes of memory each, up to thirty-two. Lakebase watches three signals — CPU load, memory usage, and the working-set size, which is how much hot data it is keeping in cache — and it adjusts capacity within the range you set, with no database restart while it stays in that range. You set two guardrails: a minimum CU, which is your responsiveness floor when traffic is low, and a maximum CU, which is your cost ceiling for spikes. The spread between them is bounded so scaling stays predictable — confirm the current cap in the docs, as it is version-dependent. And you pair it with scale-to-zero: after an idle timeout the compute suspends entirely, which is where the roughly seventy percent savings on bursty and development workloads comes from, and it resumes at the minimum size on the next connection.

CUE: Sets up the branching + cost slides: a branch is storage-only until an endpoint runs, and that endpoint scales to zero.

9 Branching & budget controls

AI CoE + CDP · 2:00

SAY: One capability to call out before the demo: branching. A Lakebase branch is a git-style, copy-on-write clone of production — schema, data and functions — created in seconds, for isolated dev, test or preview, then dropped when you are done. For CDP and Infosec the cost story is the reassuring part: a branch has no compute until you attach an endpoint, each endpoint has a capped CU and scales to zero, you gate who can create branches with project ACLs, and you tag plus attach a budget policy for chargeback. Teams self-serve safely without risking prod or the bill — and you will see this open the live demo.

CUE: Ties to §05 cost controls and to the demo opener (slide 18).

10 Section 03 — Data flow

AI CoE + CDP · 0:20

SAY: Here's the reference data flow.

11 Source → Delta → Lakebase

CDP · 4:00

SAY: Left to right: source systems — policy, claims, CRM — land through Lakeflow ingest. Delta on Unity Catalog is the governed system of record: lineage, audit, everything you already trust. From there, **synced tables** mirror the curated Delta data into Lakebase. One detail worth knowing: a synced table has a primary key and an optional **timeseries key** — a monotonic column like an update timestamp — so Lakebase always holds the *latest* row per key even if updates arrive out of order. Then apps and agents read and write live state at low latency, and you can optionally write agent state back to Delta for analytics and retention.

CUE: CDP owns this slide. Flag that "latest-row-wins" is a correctness setting, not a cost setting — cost comes two slides on.

12 Acceleration layer, not system of record

AI CoE · 3:00

SAY: Reinforcing the boundary because it drives every design decision. Lakebase *is* a serving store, operational agent memory, app and session state, a governed mirror of Delta. It is *not* your source of truth, not a BI warehouse, not where you retain long-term history, and — critically for Infosec — not a way to bypass Unity Catalog. If someone proposes making Lakebase the only home for important data, that's the anti-pattern.

CUE: Good place to pause for questions before the security section.

13 Section 04 — Security & connectivity

Infosec · 0:20

SAY: This is the part Infosec cares most about — how identity and access actually work.

14 Identity chain (documented, not live)

Infosec · 3:30

SAY: Here's the end-to-end identity chain, and it maps to your existing design doc. Entra ID authenticates the UiPath service principal. UiPath uses that to get a Databricks OAuth token for the service principal — machine-to-machine, no human in the loop. The service principal then mints a **short-lived** Lakebase database credential — a JWT that expires in about an hour — so there are **no static passwords** anywhere. That JWT maps to a specific Postgres role. So the identity you manage in Entra is the identity that shows up in the database. We're presenting this as reference architecture today; the live demo runs the same mechanics on the field-eng workspace.

CUE: Reference the Suncorp identity doc by name if it's in the room. Emphasize "short-lived, no static secrets."

15 UC / Postgres RBAC boundary

Infosec · 3:30

SAY: There are two governance planes, and the same identity is governed on both. Unity Catalog governs the lakehouse side — catalogs, schemas, lineage, the CDF history tables, Databricks-level grants and audit. Postgres RBAC governs the live rows in Lakebase — roles, row-level security, grants, the security-definer functions, and the Data API role mapping. The takeaway: this isn't a governance gap you bolt onto later; both planes are first-class, and a principal is controlled on each side.

CUE: Set up the next slide: "within the Postgres plane, the key concept is role versus ownership."

16 Role vs ownership

Infosec + AI CoE · 4:00

SAY: This is the concept that makes the demo make sense. The **authenticated principal** — the service principal or user — maps to a Postgres role that has least privilege: it can only `EXECUTE` a handful of approved functions, and it's recorded in every audit row. Separately, an **owner** role owns the tables and those functions. The functions run as the owner — that's `SECURITY DEFINER` — so they can do the work while the caller never gets direct access to tables. The runtime never owns anything. One field lesson baked in: to make this work over the HTTP Data API you must create the role with `databricks_create_role`, not the SDK — otherwise the API can't switch into it. We'll see all of this live.

CUE: Keep it conceptual here; the demo is the proof. Transition to cost.

17 Identity for the Data API & attribution

Infosec + AI CoE · 2:30

SAY: There are two identities on every operation, and separating them is the key idea. The first is the authenticated principal — the service principal or user in the OAuth token. That is what the Data API actually authenticates and switches into, it has least privilege (execute on the RPCs only), and it is recorded as the principal on every operation. The second is the logical agent id — a caller-supplied label for which agent is acting, like uipath.document_review.v1. It is passed as a parameter into every RPC call and written into the audit right next to the principal. The important distinction: the agent id is a claim, not a credential — the application asserts it, the database verifies the principal. So every audit row answers two questions: who authenticated, and who they were acting as. And because the agent id is just a label on the call, many agents can share one authenticated identity — which sets up the next slide.

CUE: Define principal (verified) vs agent_id (asserted). Leads into scaling to many agents.

18 Scaling to many agents

Infosec + AI CoE · 2:30

SAY: This answers the natural question: if I have five hundred agents, do I need five hundred service principals? No. The recommended pattern is one service principal per trust boundary — a runtime, a team, or a tenant — not per agent. The five hundred agents share that identity and each passes its own agent id, so you still get full per-agent attribution in the audit, with only a handful of principals and secrets to manage. You only create a service principal per agent when you genuinely need per-agent database authentication, isolation, or revocation. And to be clear about groups here: a group does not help. It does not reduce the number of identities — the members are still individual principals you have to create — and the Data API authenticates each one individually rather than the group. If you do need many principals, you automate it: create the service principals with Terraform or SCIM, then run databricks_create_role for each in a single idempotent SQL loop, granting each to the authenticator and the RPCs. Verified live on the workspace.

CUE: Recommendation: agent_id under one SP per trust boundary; per-agent SPs only for per-agent DB auth/isolation/revocation. A group reduces neither identities nor Data API work.

19 Section 05 — Lakeflow cost controls

CDP · 0:20

SAY: Cost — this one's for the CDP team.

20 Sync policy & cost attribution

CDP · 4:00

SAY: Two levers. First, the **sync policy**: a synced table runs in SNAPSHOT, TRIGGERED, or CONTINUOUS mode. Continuous is the freshest and the most expensive; triggered or scheduled is fine when batch freshness is acceptable. So you right-size the cadence to how fresh the data actually needs to be — that's the biggest cost dial. Second, **attribution**: tag your jobs, warehouses, and Lakebase so you can charge back by team or use case, and remember Lakebase scales to zero, which removes idle spend automatically. You monitor all of it through Unity Catalog system tables.

CUE: If pushed on numbers, offer a follow-up sizing exercise rather than quoting figures live.

21 Governed self-service branching

CDP + AI CoE · 2:30

SAY: This is the operating model that lets data science teams branch on their own while the platform team keeps control of compute and cost. The split is clean. The platform team owns the policy: one branch template with a capped compute size and scale-to-zero, branch creation run by a service principal, a budget policy and cost tags on the project, and CU-hours tracked in system tables. The data teams self-serve: they request a branch through automation rather than a ticket, receive an isolated clone of production, work on compute that is bounded and scales to zero, and the branch expires when the work ends. Recommendation: provision branches through a platform-owned automation that applies the governed template, so the platform team sets compute size, autoscaling, and lifecycle once and data teams self-serve within those limits.

CUE: The live demo section is a working reference of this automation (create, verify, teardown).

22 Shipping schema changes: the declarative line

CDP + AI CoE · 2:30

SAY: How schema changes ship is the declarative line. Above the line, bundle deploy reconciles the postgres_* infrastructure from version-controlled YAML: project, branches, endpoints, roles, database, and synced tables. Below the line, bundle run applies the application schema, its tables, columns, indexes and grants, as ordered idempotent migration files tracked in a schema_version table. Promotion is a change of target: the same code deploys and migrates dev, then stage, then prod, each its own Lakebase project. Recommendation: ship Lakebase changes with Declarative Automation Bundles, infrastructure as postgres_* resources and schema as versioned migrations promoted by target, and branch to validate a change, then promote the migration file rather than the branch.

CUE: Verified live: bundle deploy provisioned a dev project and bundle run applied the migrations on serverless.

23 The declarative line -- a simple example

CDP + AI CoE · 1:30

SAY: A simple example makes the split concrete. Take an orders table your application owns: you write its CREATE TABLE in a migration file and it applies below the line on bundle run. Now take a customer_segments table synced from a Delta source: you name the source and the primary key, Databricks derives the columns, and it deploys above the line as a postgres_synced_tables resource. Rule of thumb: if you write the columns, it is a migration below the line; if Databricks derives or manages them, it is a postgres_* resource above the line.

CUE: In the repo, resources/synced.postgres.yml is the above-the-line example and src/migrations/V1__schema.sql is the below-the-line one.

24 Section 06 — Live demo

SA · 0:30

SAY: Now the fun part. Everything we just described, running for real on the field-eng workspace. It's synthetic claims data and a non-production starter, but the security behaviors are real and verified.

CUE: Switch to the workspace / terminal. Have the run-sheet (below) open on a second screen.

25 Branch production — start the demo

SA · 2:00

SAY: Before we touch agent memory, watch how a developer works safely. I create a `dev` branch off production — it is copy-on-write, so it is instant and it carries the whole thing: the same schema, the four RPCs, and the seeded claim memory. I give it a small, budget-safe endpoint that scales to zero. Now I can experiment on `dev` — write a new memory row, try a change — and production never sees it. When I am done, I throw the branch away. That is how teams self-serve without risking prod or the bill.

CUE: Run-sheet step 1. Verified live: `dev` cloned 5 tables + 4 RPCs + the seeded memory; a write on `dev` left production at 2 rows.

26 Suncorp claims-center · agent memory

SA · 2:00

SAY: The setup: two agents — a UiPath document-review agent and a Databricks triage agent — and two synthetic claims in different business units. Shared memory lives in a Lakebase Postgres schema called `claims`. The rules: agents only call approved RPCs, never touch tables directly; every operation writes an append-only audit event, reads included; row changes flow to Unity Catalog through CDF; and cross-claim access is denied. Let me show you.

CUE: Begin the run-sheet. Narrate each result as it returns.

27 Data API vs direct SQL

SA · 3:00

SAY: Two ways in, same governance. The **Data API** is PostgREST over HTTPS with an OAuth bearer token — no Postgres driver needed, which is exactly what a UiPath or low-code runtime wants. **Direct SQL** is psql or psycopg with the same OAuth credential — for apps, notebooks, migrations. Both hit the same roles, the same row-level security, the same RPCs. I'll use the Data API as the UiPath agent, and direct SQL as the Databricks agent, so you can see them sharing state.

CUE: Run-sheet steps 1–2 (UiPath write via Data API; Databricks read via SQL).

28 What the demo proves

SA · 5:00

SAY: Here's the scorecard, and these are actual results. As the service principal over the Data API, read, write, and task RPCs all return 200. A cross-claim completion returns 403 — denied, and it writes nothing. A direct table read returns 403 — the runtime literally cannot bypass the RPCs. The audit trail records both the authenticated principal *and* the logical agent id, so you can tell who really called versus who they claimed to be. And the row-change history is queryable in Unity Catalog via CDF. Green is good here — including the two 403s, because denial is the point.

CUE: Run-sheet steps 3–6. Land the "403 is a feature" line.

29 Security takeaways

Host / Infosec · 2:00

SAY: To summarize the security posture: runtimes get EXECUTE on approved RPCs only — no direct table writes; row-level security is fail-closed, so direct access returns nothing or is denied; the RPCs enforce explicit claim and business-unit checks in one transaction with the audit write; audit captures the real principal and the logical agent; and CDF gives you immutable write-history in Unity Catalog, with reads audited separately in the app layer.

CUE: Invite Infosec to confirm this meets their bar, or name the gaps.

30 Recommendations & next steps

Host · 2:00

SAY: Concretely: use separate service principals per agent runtime so attribution is strong; keep Lakebase as the serving layer and Delta as the system of record; standardize the identity chain with Infosec — Entra to OAuth to Postgres role; set the sync cadence and cost tags with CDP; and review the SQL and RPC pattern with security before anything goes to production. We have a reviewed starter package that implements all of this.

CUE: Assign owners and a follow-up date live if you can.

31 **Questions & discussion** All · 2:00

SAY: That's the pattern end to end. What questions do you have — and where do you want to go next?

CUE: Use the Q&A prep below.

Live-demo run-sheet

Exact sequence (slides 18–22)

1. **Branch production (the opener).** `databricks postgres create-branch projects/suncorp-claims-center dev --json '{"spec": {"source_branch":".../branches/production","no_expiry":true}}'`. Show it is READY in seconds. Connect to the `dev` endpoint and show it already has the `claims` schema, the 4 RPCs, and the seeded memory (copy-on-write clone). Write a throwaway row on `dev` (`rpc_write_memory('CLM-10001',...,'dev.experiment',...)`), then show **production is unchanged** (still 2 memory rows, no `dev.experiment`). Note the endpoint inherits the small 1 CU cap and scales to zero. Finish with `delete-branch .../dev` — gone, storage reclaimed. *(Verified live end-to-end.)*
2. **Show the shape.** Back on production, briefly show the `claims` schema + the four RPCs (`rpc_read_memory`, `rpc_write_memory`, `rpc_create_task`, `rpc_complete_task`). Point out: no table grants to the agents.
3. **UiPath agent · Data API (HTTPS + OAuth).** As the service principal, `rpc_write_memory` + `rpc_create_task` on `CLM-10001`. Expect **200 OK**. Narrate: "no Postgres driver — just a bearer token."
4. **Databricks agent · direct SQL.** `rpc_read_memory` for `CLM-10001` — returns the memory the UiPath agent just wrote. Narrate: "two runtimes, one governed memory."
5. **Negative — cross-claim.** Same SP tries `rpc_complete_task` on a `CLM-10001` task while asserting `CLM-10002` → **403 — cross-claim access denied**, zero writes.
6. **Negative — direct table.** Same SP tries `SELECT * FROM claims.agent_memory` → **403 — permission denied for table**. "The RPCs are the only door."
7. **Audit + CDF.** Show `audit_event` rows: `principal` = the SP app-id, `agent_id` = the logical agent. Then show the CDF history table in Unity Catalog (`suncorp_cdf.suncorp_claims_cdf`) returning the row changes.

Fallback: if the Data API round-trip has any hiccup live, run the same RPCs over direct SQL as the SP — the boundary behavior (200s and 403s) is identical and already verified.

Q&A prep — likely questions

Is Lakebase our new system of record?

No. Delta on Unity Catalog remains the source of truth. Lakebase is the low-latency serving/acceleration layer; anything durable is retained in Delta.

How does a UiPath bot authenticate — do we store a password?

No static secrets. Entra authenticates the service principal, which gets a Databricks OAuth token and mints a short-lived (~1h) Lakebase credential that maps to a Postgres role.

Can an agent read another claim's data?

No. The RPCs enforce explicit claim/business-unit checks and deny cross-claim calls (demonstrated: 403). Direct table access is also denied by row-level security + least-privilege grants.

What controls cost?

Mainly the sync policy — SNAPSHOT/TRIGGERED/CONTINUOUS — matched to required freshness, plus scale-to-zero on idle endpoints. Tag jobs/warehouses/Lakebase for chargeback; monitor via UC system tables.

What's the timeseries key?

A synced-table setting: a monotonic column (e.g. update timestamp) that keeps the latest row per primary key when updates arrive out of order. It's a correctness setting on the Delta→Lakebase mirror — not used by the agent's own write-path memory, which upserts directly.

Where do we see what an agent did, and who really did it?

Every operation writes an append-only audit event with the authenticated principal and the logical agent id. Writes also appear in Unity Catalog CDF; reads are captured in the app audit, not CDF.

Why RPCs instead of just granting table access?

RPCs let business logic and the audit write commit in one transaction, keep least privilege (EXECUTE only), and enforce claim boundaries centrally. Runtimes never hold table DML.

What happens on scale-to-zero — is there a cold start?

Idle endpoints scale to zero and wake on the next connection; there's a brief wake latency. For always-hot paths, set a minimum CU above zero.